

MIS 6326: Open Test

14th September 2025.

Name: Mugizi Polycarp

Registration Number: 2024-M132-24001

1. First EDA (Raw Data)

- Summary statistics of tumor attributes (mean, std, IQR).

Code file: summary_stats.py

```
# Read dataset from url
df = pd.read_csv(url, names=columns)

# Replace missing values "?" with NaN and convert to numeric
df["bare_nuclei"] = pd.to_numeric(df["bare_nuclei"], errors="coerce")

# Drop ID and Class (not tumor attributes)
tumor_attributes = df.drop(columns=["id", "class"])

# Compute summary statistics
summary_stats = pd.DataFrame({
    "Mean": tumor_attributes.mean(),
    "StdDev": tumor_attributes.std(),
    "IQR": tumor_attributes.quantile(0.75) - tumor_attributes.quantile(0.25)
})

print(summary_stats)
```

Output

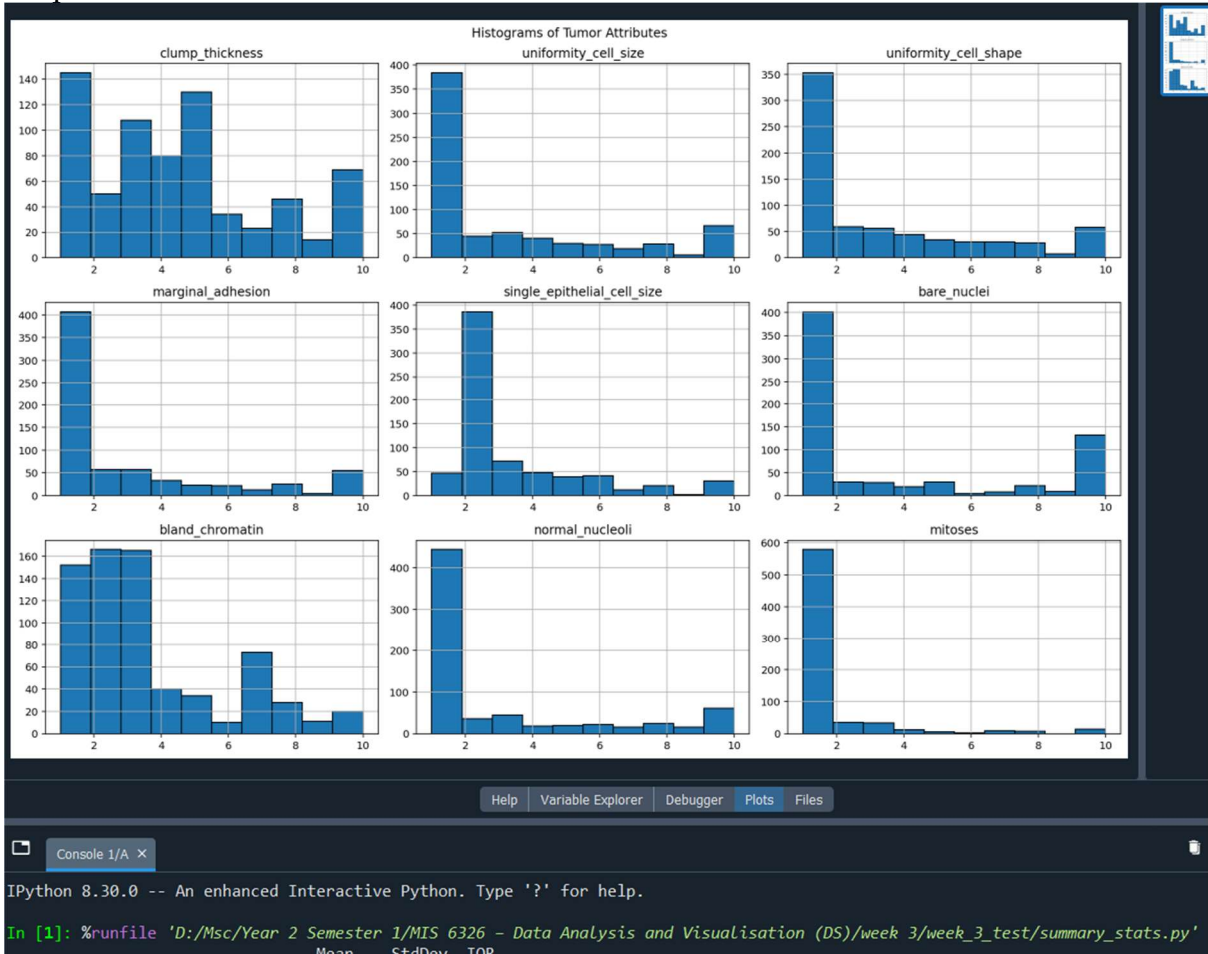
```
In [1]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/
week_3_test/summary_stats.py' --wdir
          Mean    StdDev  IQR
clump_thickness    4.417740  2.815741  4.0
uniformity_cell_size    3.134478  3.051459  4.0
uniformity_cell_shape    3.207439  2.971913  4.0
marginal_adhesion    2.806867  2.855379  3.0
single_epithelial_cell_size    3.216023  2.214300  2.0
bare_nuclei    3.544656  3.643857  5.0
bland_chromatin    3.437768  2.438364  3.0
normal_nucleoli    2.866953  3.053634  3.0
mitoses    1.589413  1.715078  0.0
```

- Histograms for each attribute.

Code file: histogram.py

```
# Plot histograms for each attribute
tumor_attributes.hist(bins=10, figsize=(15, 10), edgecolor="black", grid=True)
plt.suptitle("Histograms of Tumor Attributes", fontsize=12)
plt.tight_layout()
plt.show()
```

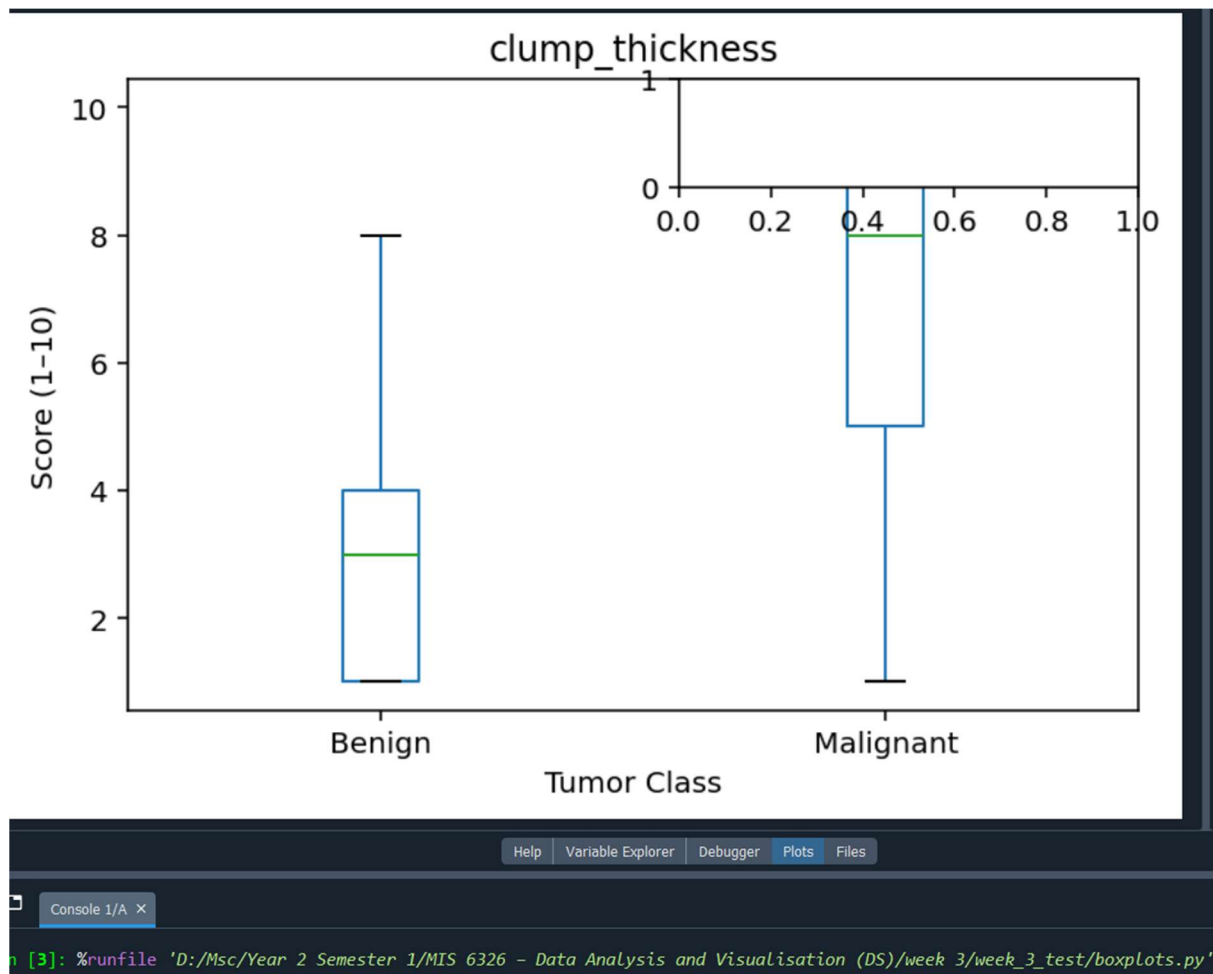
Output

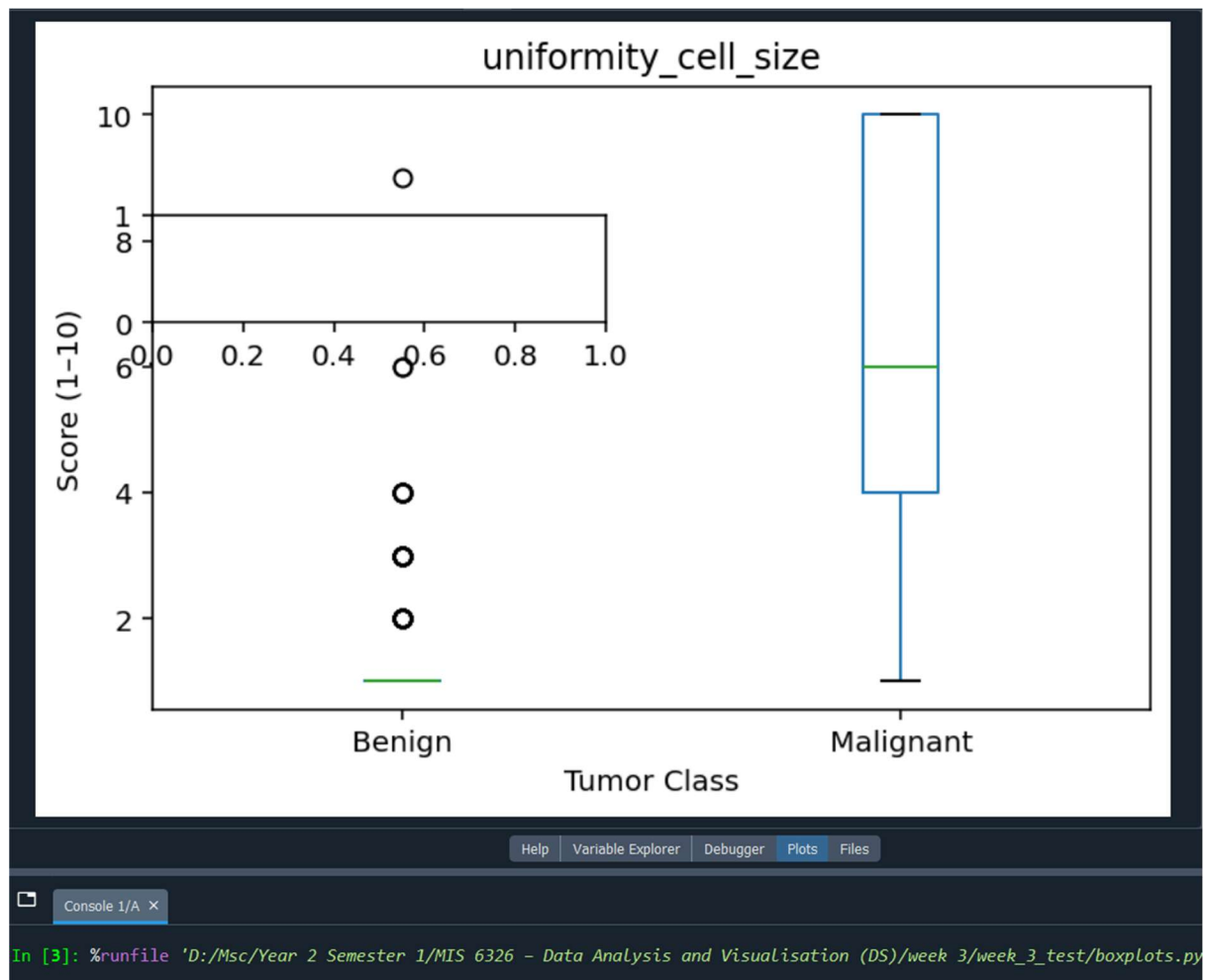


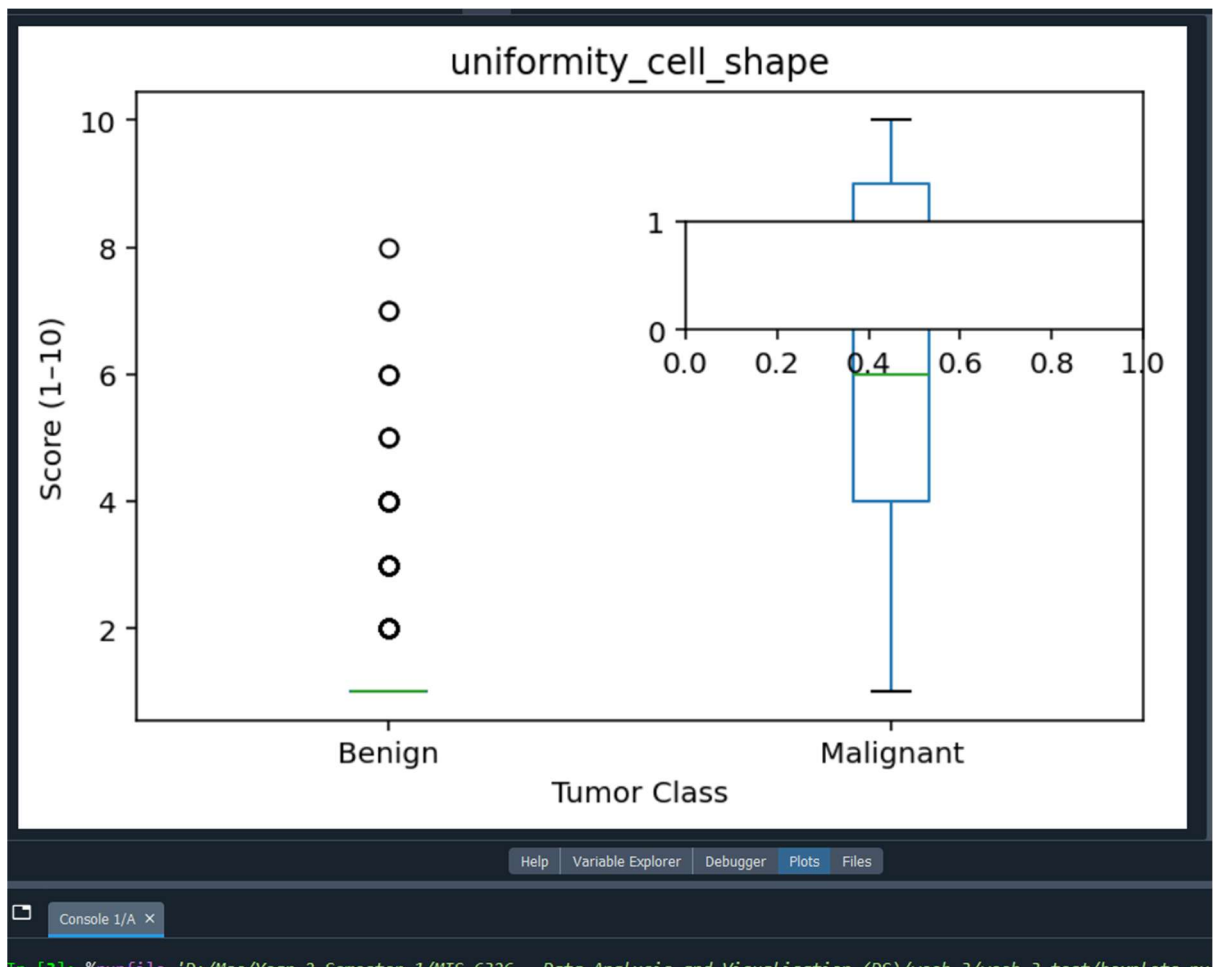
- Boxplots comparing benign vs malignant tumors.

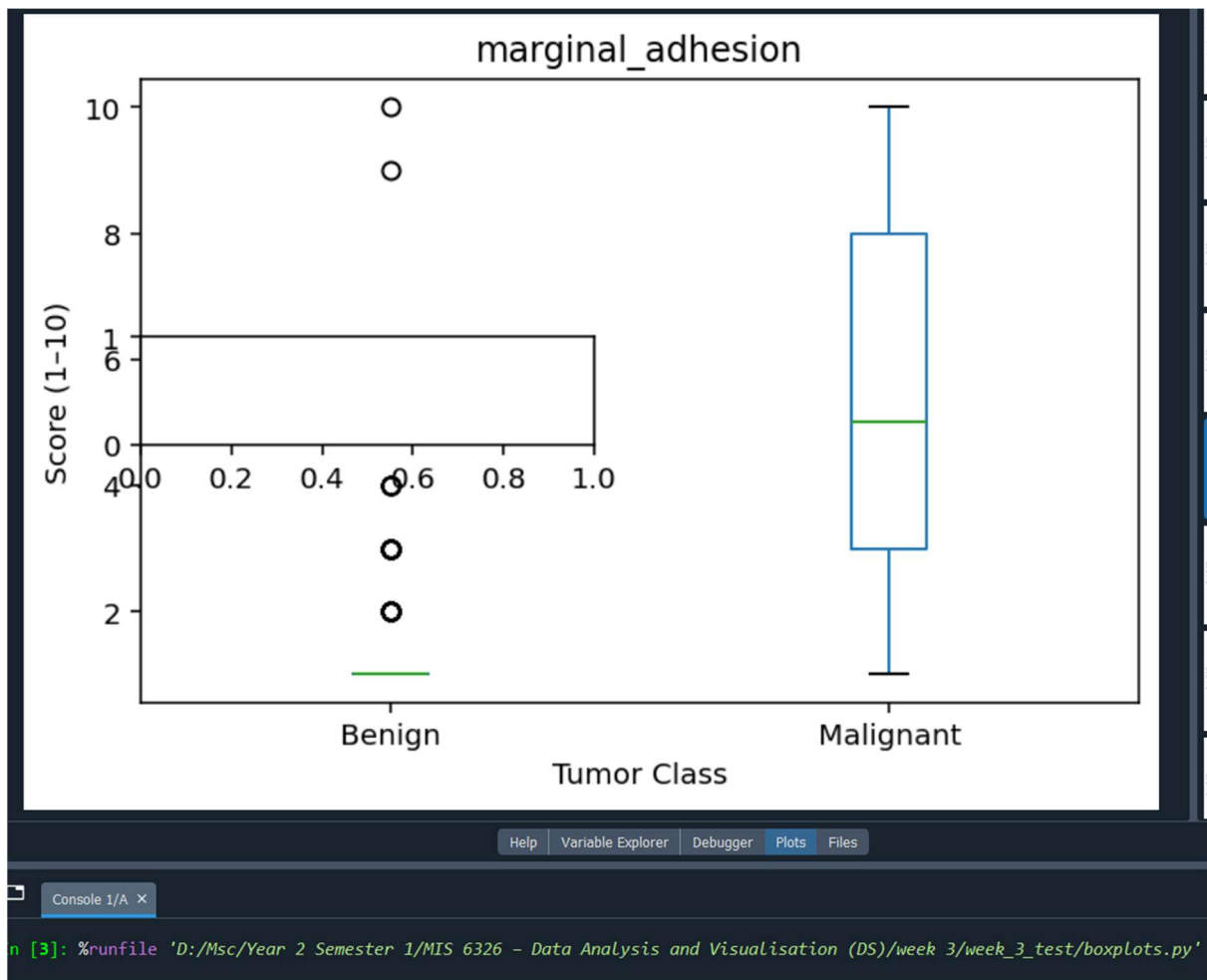
Code file: boxplots.py

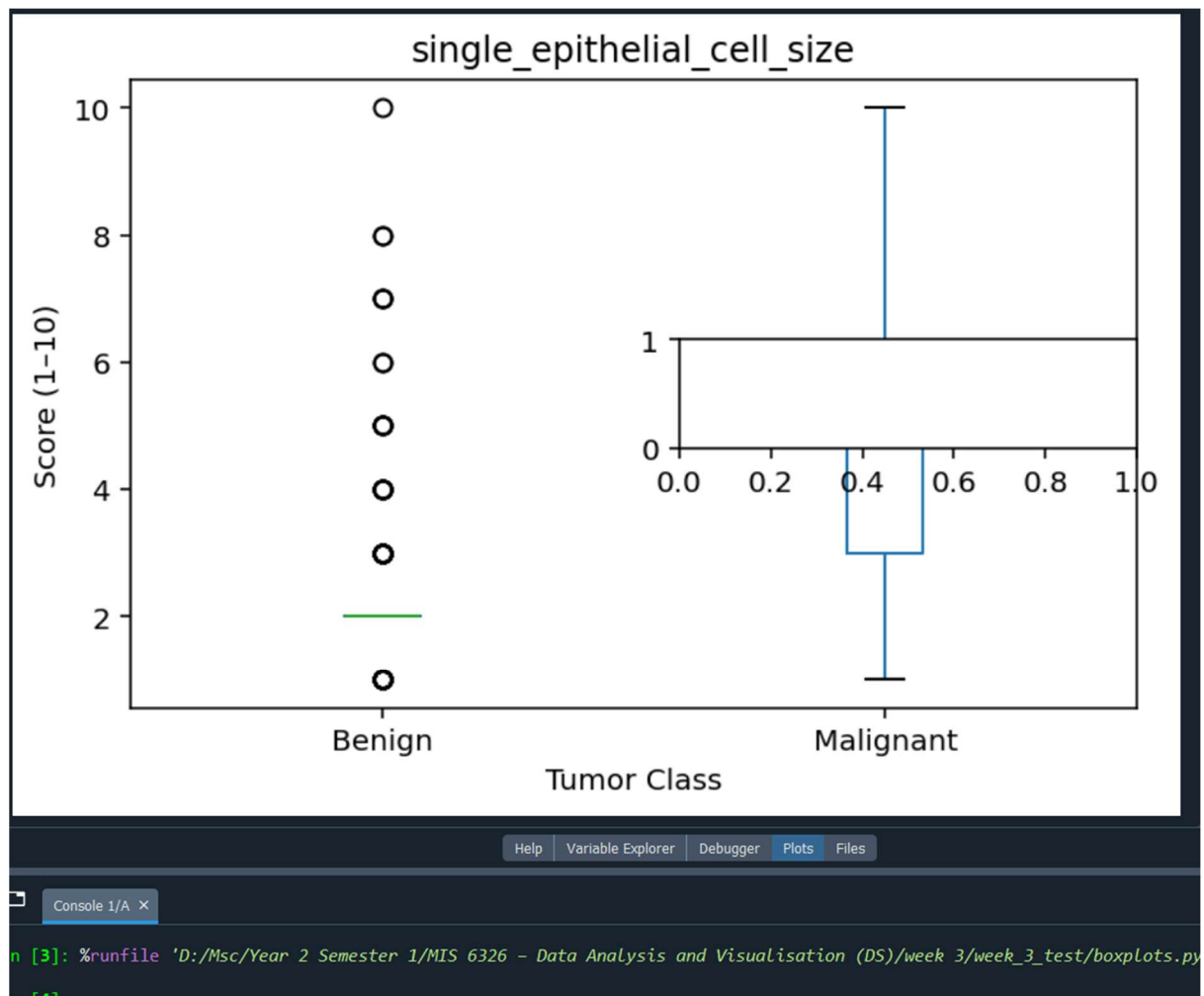
```
39 # Plot boxplots for each attribute grouped by class
40 plt.figure(figsize=(10, 15))
41 for i, col in enumerate(attributes.columns[:-1, 1]): # skip class column
42     plt.subplot(5, 2, i)
43     attributes.boxplot(column=col, by="class", grid=False)
44     plt.title(col)
45     plt.suptitle("") # remove automatic title
46     plt.xlabel("Tumor Class")
47     plt.ylabel("Score (1-10)")
48
49 plt.tight_layout() # Adjust layout so titles/labels don't overlap
50 plt.show() # Display
51
```

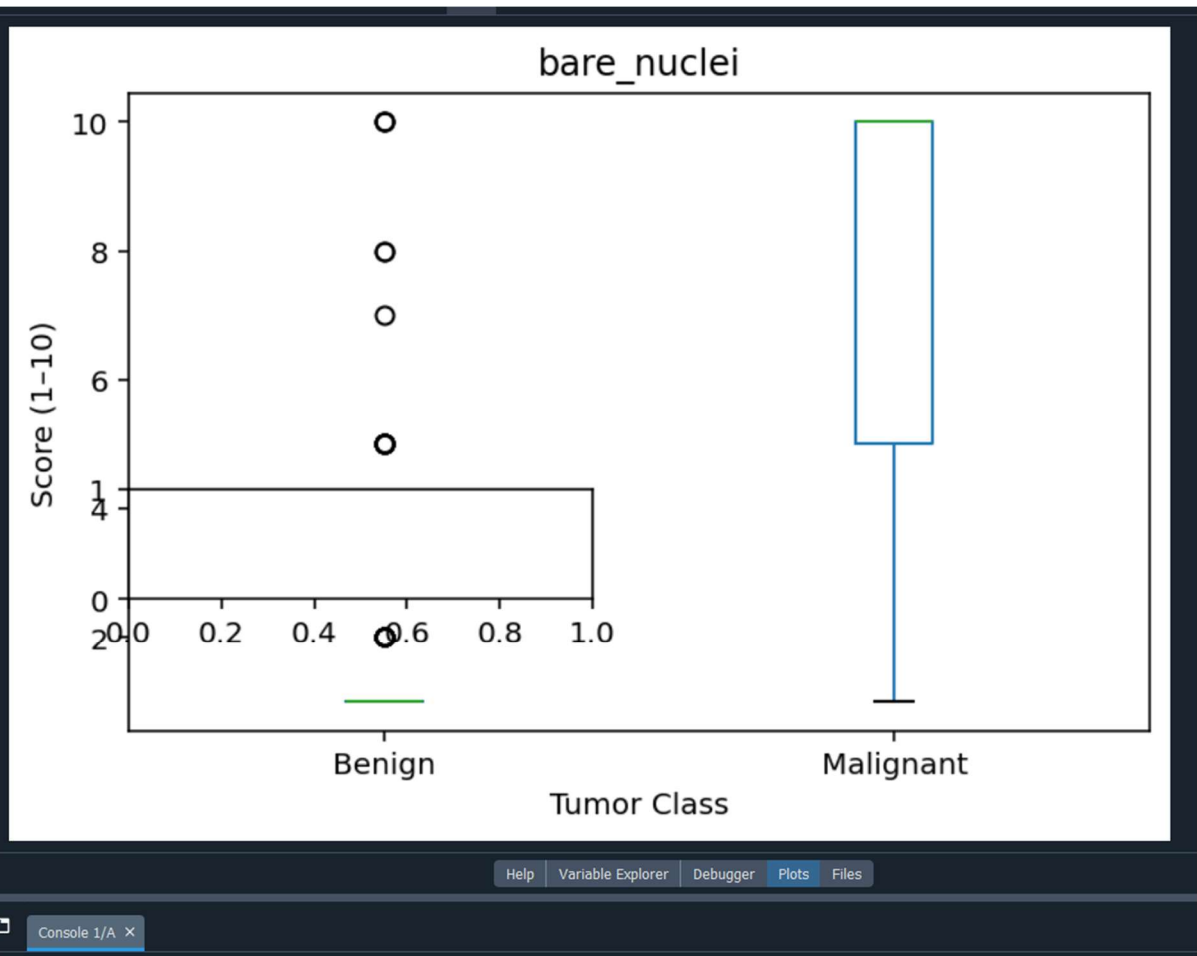


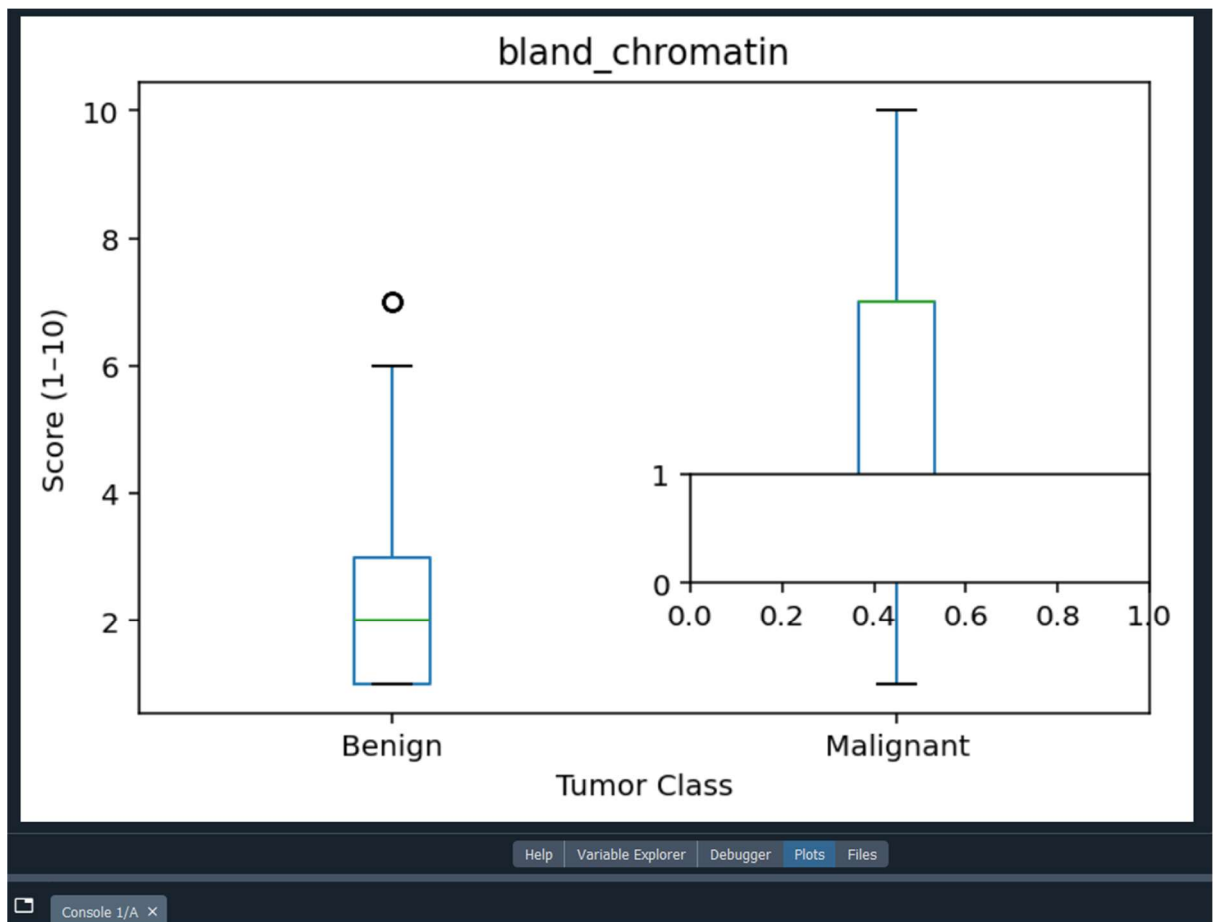


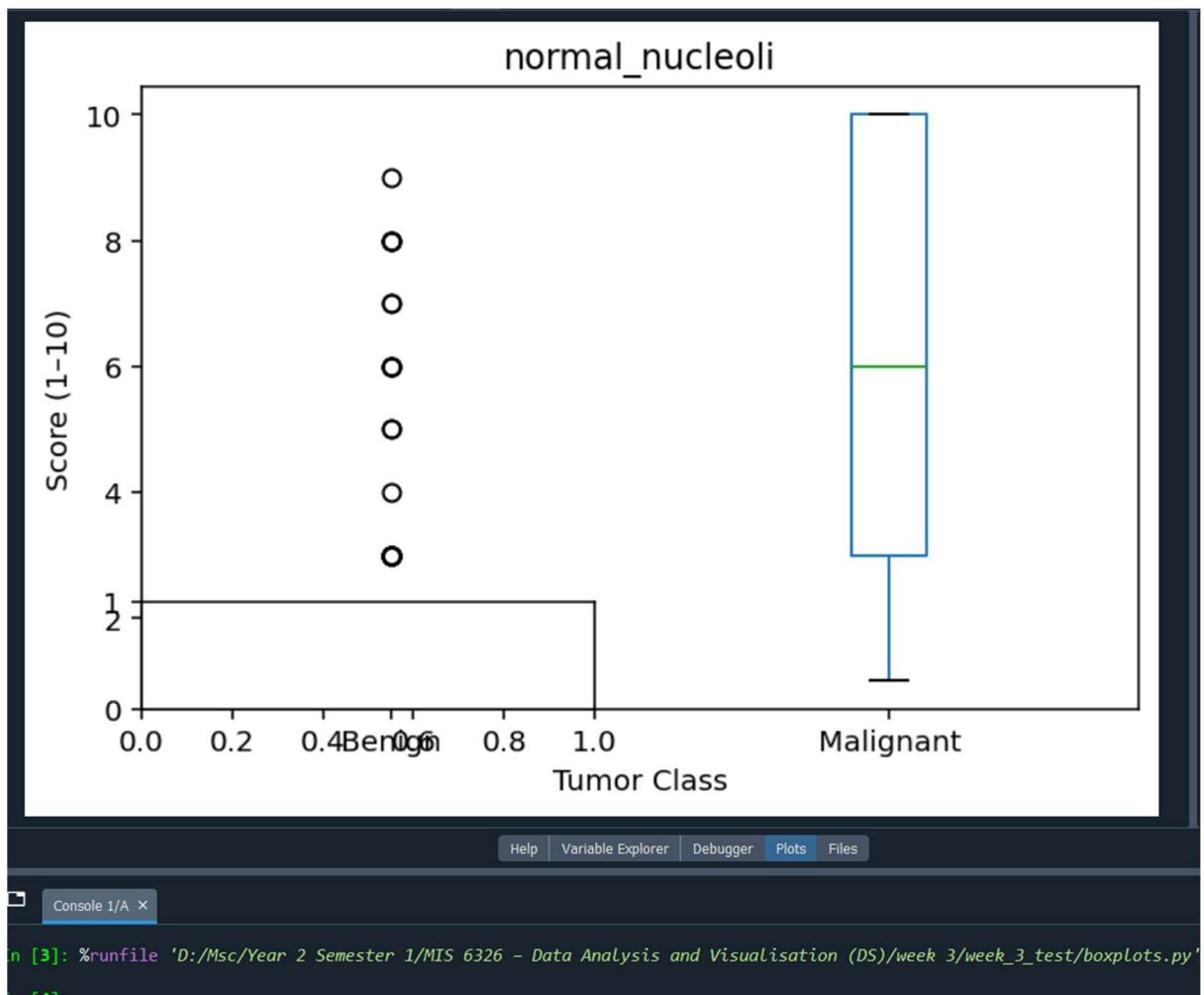


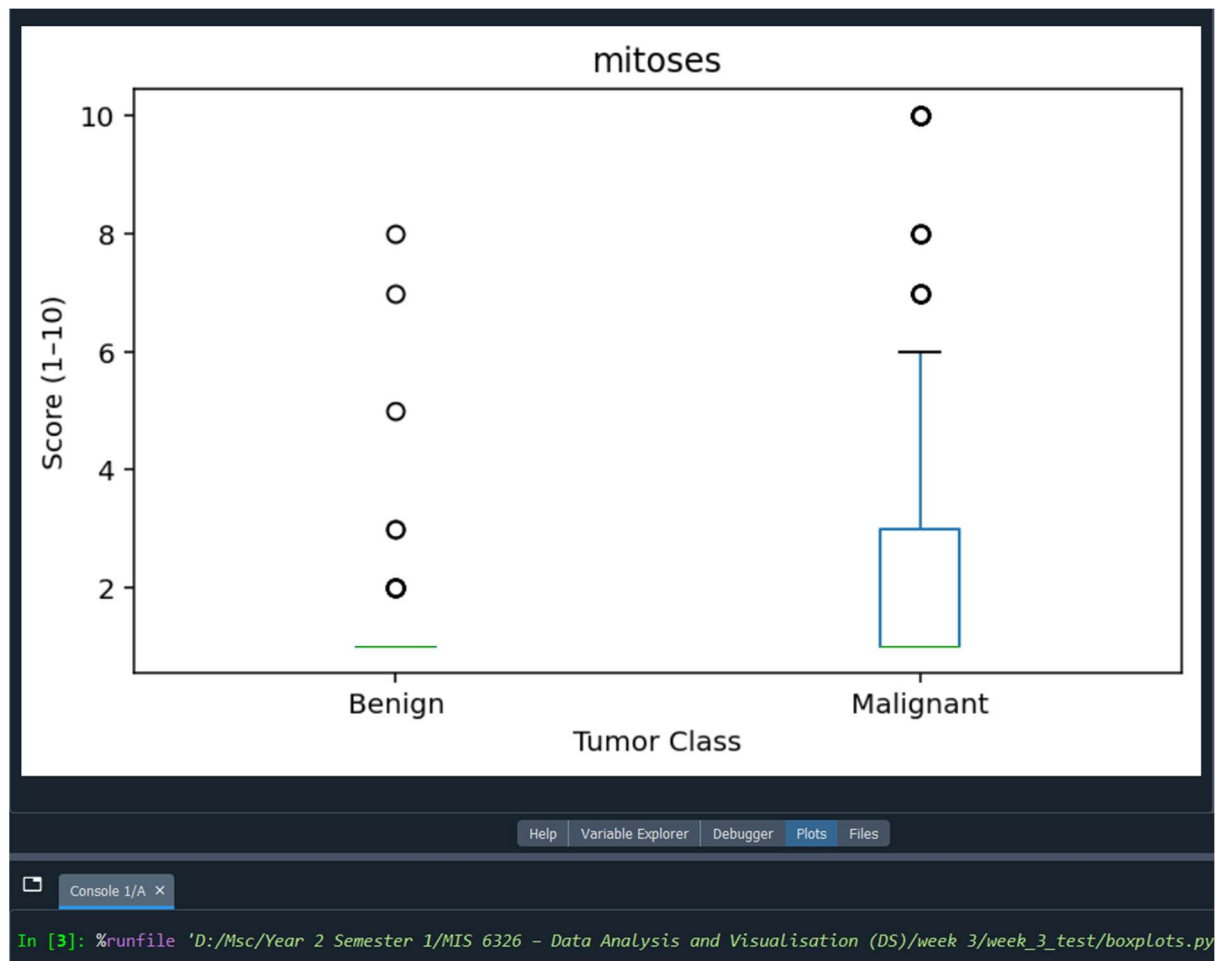












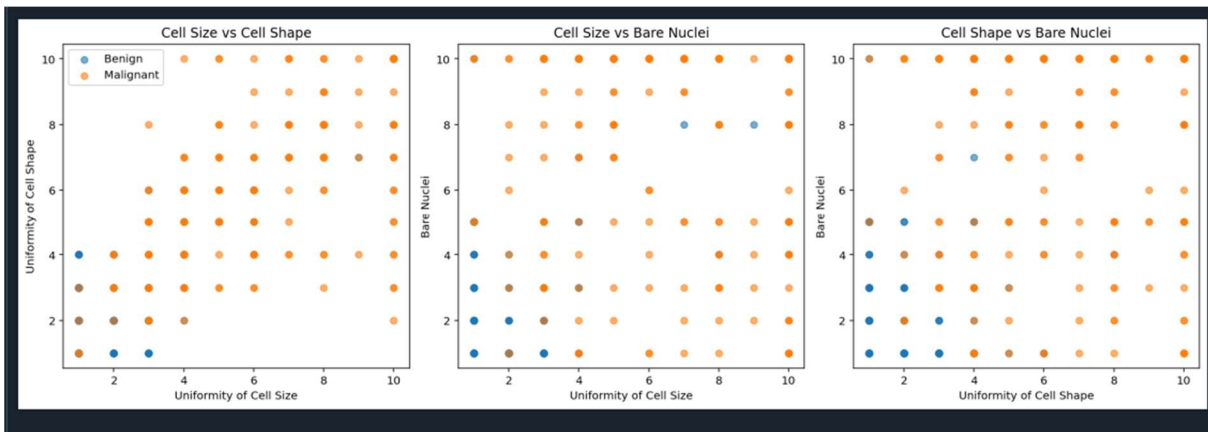
- Scatter plots between top 2–3 features.

Code file: scatterplots.py

```

45 # -----
46 # | Scatter Plots for Top Features
47 # -----
48 plt.figure(figsize=(15, 5)) # make the figure wide enough for 3 plots
49
50 # --- (a) Uniformity of cell size vs Uniformity of cell shape ---
51 plt.subplot(1, 3, 1) # 1 row, 3 columns, first subplot
52 for label, group in df.groupby("class"): # plot each class separately
53     plt.scatter(group["uniformity_cell_size"], group["uniformity_cell_shape"],
54                 label=label, alpha=0.6) # alpha for transparency
55 plt.xlabel("Uniformity of Cell Size")
56 plt.ylabel("Uniformity of Cell Shape")
57 plt.title("Cell Size vs Cell Shape")
58 plt.legend()
59
60 # --- (b) Uniformity of cell size vs Bare nuclei ---
61 plt.subplot(1, 3, 2) # second subplot
62 for label, group in df.groupby("class"):
63     plt.scatter(group["uniformity_cell_size"], group["bare_nuclei"],
64                 label=label, alpha=0.6)
65 plt.xlabel("Uniformity of Cell Size")
66 plt.ylabel("Bare Nuclei")
67 plt.title("Cell Size vs Bare Nuclei")
68
69 # --- (c) Uniformity of cell shape vs Bare nuclei ---
70 plt.subplot(1, 3, 3) # third subplot
71 for label, group in df.groupby("class"):
72     plt.scatter(group["uniformity_cell_shape"], group["bare_nuclei"],
73                 label=label, alpha=0.6)
74 plt.xlabel("Uniformity of Cell Shape")
75 plt.ylabel("Bare Nuclei")
76 plt.title("Cell Shape vs Bare Nuclei")
77
78 # Adjust layout so titles/labels don't overlap
79 plt.tight_layout()
80
81 # Display all 3 scatter plots
82 plt.show()
83

```



1. Preprocessing
 - Handle missing values.

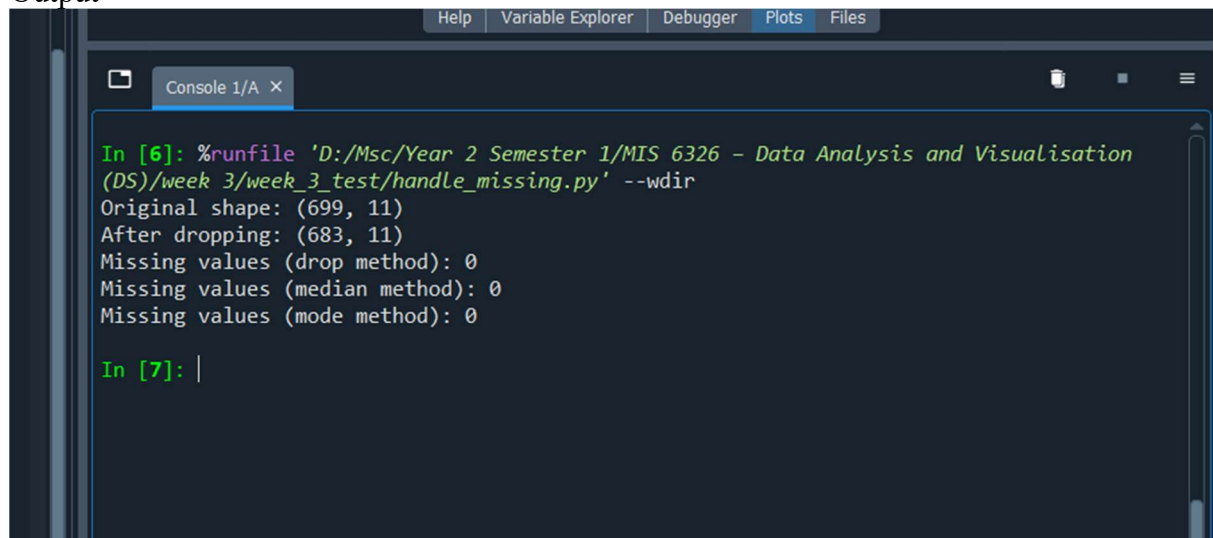
Code file: `handle_missing.py`

```

30 # Load the dataset into a DataFrame
31 df = pd.read_csv(url, names=columns)
32
33 # -----
34 # 2. Handle missing values
35 # -----
36 # The 'bare_nuclei' column has missing entries marked with '?'
37 # Convert the column to numeric, turning '?' into NaN automatically
38 df["bare_nuclei"] = pd.to_numeric(df["bare_nuclei"], errors="coerce")
39
40 # -----
41 # Option A: Drop rows with missing values
42 # -----
43 df_dropped = df.dropna() # removes any row containing NaN
44 print("Original shape:", df.shape) # should be (699, 11)
45 print("After dropping:", df_dropped.shape) # should be (683, 11)
46
47 # -----
48 # Option B: Impute missing values with the median
49 # -----
50 # Calculate the median of 'bare_nuclei' and replace NaN with it
51 df_median = df.copy()
52 df_median["bare_nuclei"] = df_median["bare_nuclei"].fillna(df_median["bare_nuclei"].median())
53
54 # -----
55 # Option C: Impute missing values with the mode
56 # -----
57 # Calculate the most frequent value (mode) and use it for NaN
58 df_mode = df.copy()
59 df_mode["bare_nuclei"] = df_mode["bare_nuclei"].fillna(df_mode["bare_nuclei"].mode()[0])
60
61 # -----
62 # 3. Verify cleaning
63 # -----
64 print("Missing values (drop method):", df_dropped.isna().sum().sum())
65 print("Missing values (median method):", df_median.isna().sum().sum())
66 print("Missing values (mode method):", df_mode.isna().sum().sum())
67

```

Output



```

In [6]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation
(DS)/week 3/week_3_test/handle_missing.py' --wdir
Original shape: (699, 11)
After dropping: (683, 11)
Missing values (drop method): 0
Missing values (median method): 0
Missing values (mode method): 0

In [7]: |

```

- Normalize continuous features.

Code file: normalize.py

```

50
51 # -----
52 # 4a. Min-Max Normalization (scales values to range [0,1])
53 # -----
54 # Create a MinMaxScaler object
55 minmax_scaler = MinMaxScaler()
56
57 # Fit the scaler to the features and transform them
58 features_minmax = minmax_scaler.fit_transform(features)
59
60 # Convert back into a DataFrame with the same column names
61 df_minmax = pd.DataFrame(features_minmax, columns=features.columns)
62
63 print("First 5 rows after Min-Max Normalization:")
64 print(df_minmax.head(), "\n")
65
66 # -----
67 # 4b. Z-Score Standardization (mean=0, std=1). transforms features to have mean = 0 and std = 1
68 # -----
69 # Create a StandardScaler object
70 zscore_scaler = StandardScaler()
71
72 # Fit the scaler to the features and transform them
73 features_zscore = zscore_scaler.fit_transform(features)
74
75 # Convert back into a DataFrame with the same column names
76 df_zscore = pd.DataFrame(features_zscore, columns=features.columns)
77
78 print("First 5 rows after Z-Score Standardization:")
79 print(df_zscore.head())
80

```

Output

```

Console 1/A x
Missing values (median method): 0
Missing values (mode method): 0

In [7]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation
(DS)/week 3/week_3_test/normalize.py' --wdir
First 5 rows after Min-Max Normalization:
   clump_thickness  uniformity_cell_size  ...  normal_nucleoli  mitoses
0         0.444444         0.000000  ...         0.000000         0.0
1         0.444444         0.333333  ...         0.111111         0.0
2         0.222222         0.000000  ...         0.000000         0.0
3         0.555556         0.777778  ...         0.666667         0.0
4         0.333333         0.000000  ...         0.000000         0.0

[5 rows x 9 columns]

First 5 rows after Z-Score Standardization:
   clump_thickness  uniformity_cell_size  ...  normal_nucleoli  mitoses
0         0.197905        -0.702212  ...        -0.612927        -0.3484
1         0.197905         0.277252  ...        -0.285105        -0.3484
2        -0.511643        -0.702212  ...        -0.612927        -0.3484
3         0.552679         1.583204  ...         1.354008        -0.3484
4        -0.156869        -0.702212  ...        -0.612927        -0.3484

[5 rows x 9 columns]

In [8]:

```

- Encode categorical features if present.

Code file: encode.py

```

44 # -----
45 # 3. Encode categorical features
46 # -----
47 # 'class' is categorical with numeric codes: 2 = Benign, 4 = Malignant
48 # Most ML models prefer binary encoding: 0 or 1
49 # Map values: 2 -> 0 (Benign), 4 -> 1 (Malignant)
50 df["class"] = df["class"].map({2: 0, 4: 1})
51
52 # -----
53 # 4. Verify encoding
54 # -----
55 # Check the number of samples in each class after encoding
56 print(df["class"].value_counts())
57

```

Output

```

In [8]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation
(DS)/week 3/week_3_test/encode.py' --wdir
class
0      444
1      239
Name: count, dtype: int64

```

2. Feature Engineering & Reduction

- Generate ratios (e.g., cell size/shape).

Code file: ratios.py

```

50 # 4. Generate feature ratios
51 # -----
52 # Ratios can highlight relationships between attributes that raw values may not capture
53
54 # Add small epsilon (1e-6) to avoid division by zero
55 epsilon = 1e-6
56
57 # Ratio 1: Uniformity of cell size / Uniformity of cell shape
58 df["size_shape_ratio"] = df["uniformity_cell_size"] / (df["uniformity_cell_shape"] + epsilon)
59
60 # Ratio 2: Clump thickness / Uniformity of cell size
61 df["thickness_size_ratio"] = df["clump_thickness"] / (df["uniformity_cell_size"] + epsilon)
62
63 # Ratio 3: Bare nuclei / Normal nucleoli
64 df["bare_nuclei_nucleoli_ratio"] = df["bare_nuclei"] / (df["normal_nucleoli"] + epsilon)
65
66 # -----
67 # 5. Inspect new features
68 # -----
69 # Display first few rows of the newly created ratio features
70 print(df[["size_shape_ratio", "thickness_size_ratio", "bare_nuclei_nucleoli_ratio"]].head())
71

```

Output

```

In [9]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation
(DS)/week 3/week_3_test/ratios.py' --wdir
size_shape_ratio  thickness_size_ratio  bare_nuclei_nucleoli_ratio
0      0.999999      4.999995      0.999999
1      1.000000      1.250000      4.999998
2      0.999999      2.999997      1.999998
3      1.000000      0.750000      0.571428
4      0.999999      3.999996      0.999999

In [10]:

```

- Apply Recursive Feature Elimination (RFE).

Code file: RFE.py

```
36 # Create ratio features
37 # -----
38 epsilon = 1e-6
39 df["size_shape_ratio"] = df["uniformity_cell_size"] / (df["uniformity_cell_shape"] + epsilon)
40 df["thickness_size_ratio"] = df["clump_thickness"] / (df["uniformity_cell_size"] + epsilon)
41 df["bare_nuclei_nucleoli_ratio"] = df["bare_nuclei"] / (df["normal_nucleoli"] + epsilon)
42
43 # -----
44 # 5. Prepare feature matrix X and target y
45 # -----
46 # Exclude 'id' and 'class'
47 X = df.drop(columns=["id", "class"])
48 y = df["class"]
49
50 # -----
51 # 6. Normalize features (important for some models)
52 # -----
53 scaler = StandardScaler()
54 X_scaled = scaler.fit_transform(X)
55
56 # -----
57 # 7. Apply RFE using Logistic Regression
58 # -----
59 # Create logistic regression model
60 model = LogisticRegression(max_iter=1000)
61
62 # RFE: select top 5 features
63 rfe = RFE(estimator=model, n_features_to_select=5)
64 rfe.fit(X_scaled, y)
65
66 # -----
67 # 8. Output results
68 # -----
69 # Boolean mask of selected features
70 print("Selected features (True = chosen):")
71 print(pd.Series(rfe.support_, index=X.columns))
72
73 # Feature ranking (1 = most important)
74 print("\nFeature ranking:")
75 print(pd.Series(rfe.ranking_, index=X.columns))
76
```

Output

```
In [10]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/RFE.py' --wdir
Selected features (True = chosen):
clump_thickness      True
uniformity_cell_size False
uniformity_cell_shape True
marginal_adhesion    False
single_epithelial_cell_size False
bare_nuclei          False
bland_chromatin      True
normal_nucleoli       True
mitoses              False
size_shape_ratio     False
thickness_size_ratio  False
bare_nuclei_nucleoli_ratio True
dtype: bool

Feature ranking:
clump_thickness      1
uniformity_cell_size 7
uniformity_cell_shape 1
marginal_adhesion    2
single_epithelial_cell_size 6
bare_nuclei          4
bland_chromatin      1
normal_nucleoli       1
mitoses              3
size_shape_ratio     8
thickness_size_ratio  5
bare_nuclei_nucleoli_ratio 1
dtype: int64

In [11]:
```

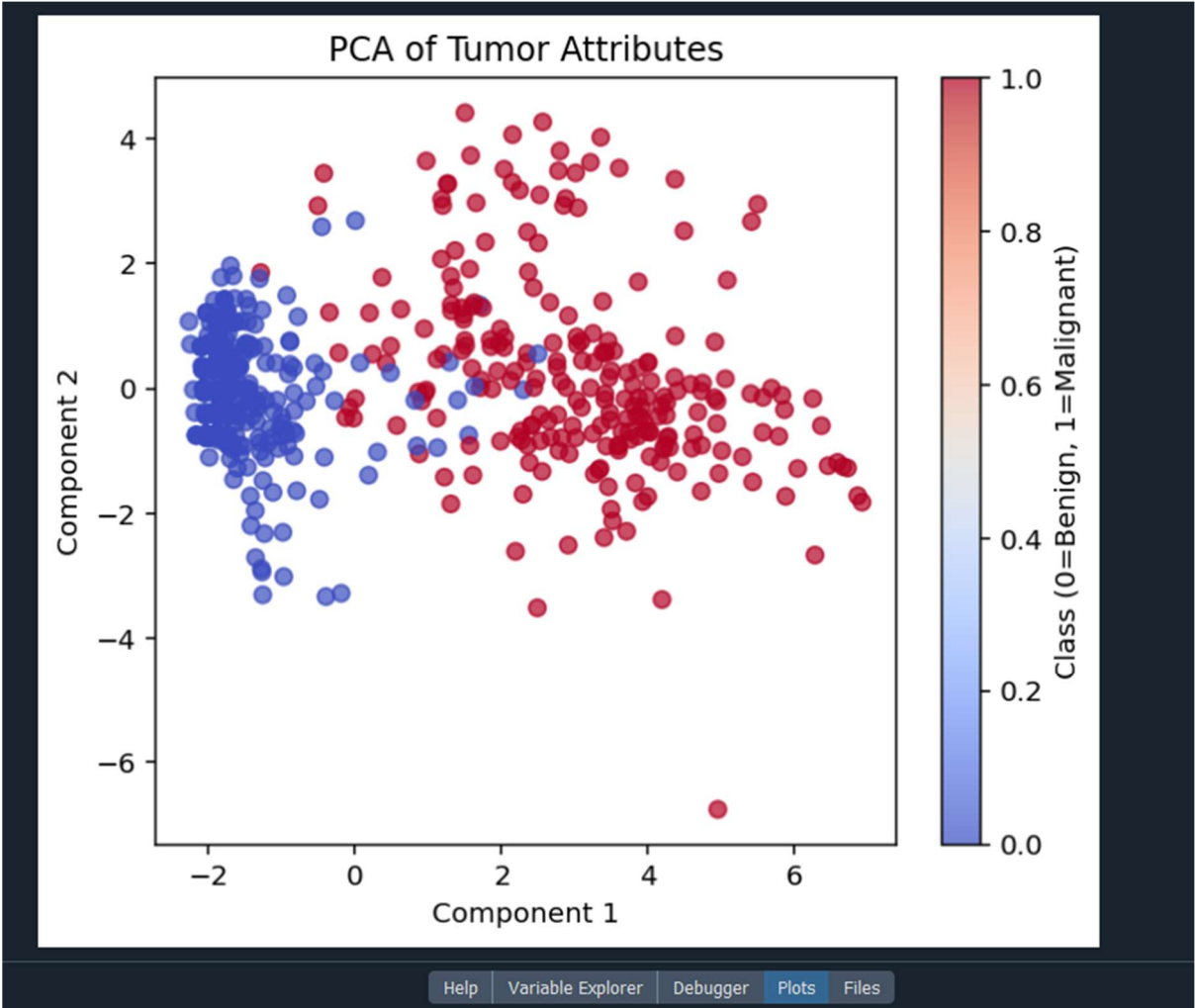
- Run PCA, t-SNE, and UMAP to capture variance in fewer components.

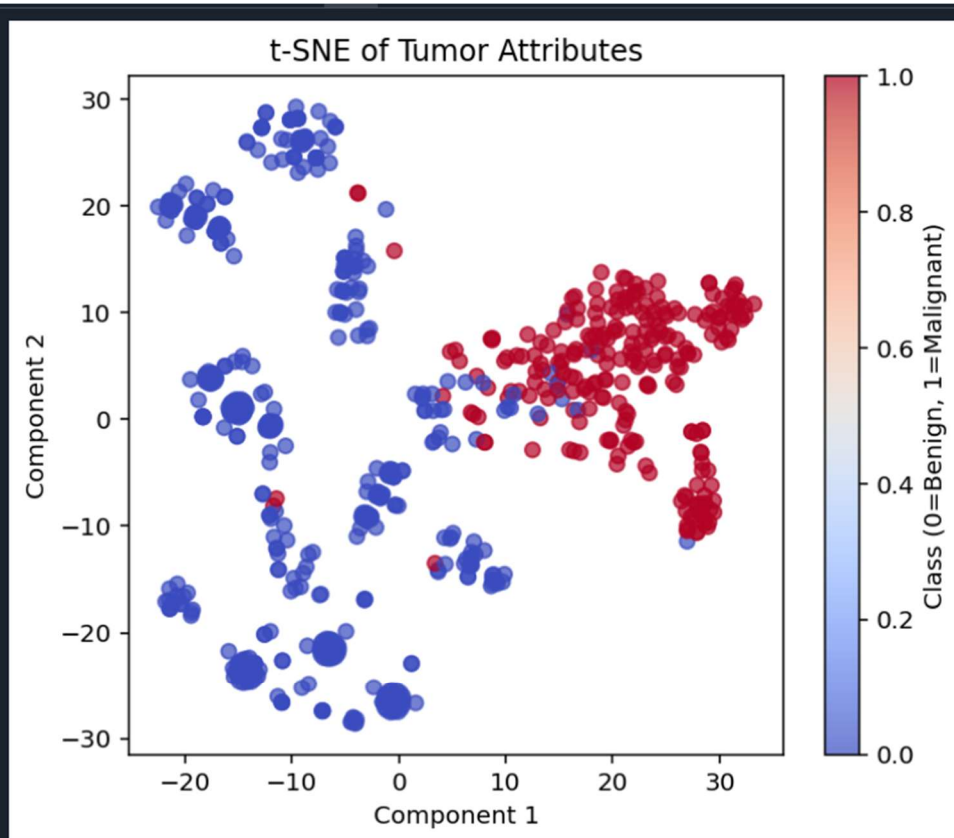
Code file: PCA t-SNE UMAP.py

```
PCA_t-SNE_UMAP.py x
88 # 7a. PCA (Principal Component Analysis)
89 # PCA is a linear dimensionality reduction technique #We reduce to 2 components for visualization
90 pca = PCA(n_components=2)
91 X_pca = pca.fit_transform(X_scaled)
92
93 # Check how much variance each component explains
94 print("Explained variance ratio by PCA:", pca.explained_variance_ratio_)
95
96 # 7b. t-SNE (t-distributed Stochastic Neighbor Embedding)
97 # t-SNE is a non-linear technique, preserves local structure
98 # Perplexity controls balance between local/global structure
99 tsne = TSNE(n_components=2, random_state=42, perplexity=30)
100 X_tsne = tsne.fit_transform(X_scaled)
101
102 # 7c. UMAP (Uniform Manifold Approximation and Projection)
103 # UMAP is a non-linear technique preserving both local and global structure
104 umap_reducer = umap.UMAP(n_components=2, random_state=42)
105 X_umap = umap_reducer.fit_transform(X_scaled)
106
107 # 8. Visualize 2D embeddings
108 def plot_2d(X_embedded, title):
109     """
110     Plot 2D embedding with color by tumor class
111     X_embedded: 2D array of shape (n_samples, 2)
112     title: plot title
113     """
114     plt.figure(figsize=(6,5))
115     plt.scatter(
116         X_embedded[:,0], X_embedded[:,1],
117         c=y, # color by class
118         cmap='coolwarm', # color map
119         alpha=0.7 # point transparency
120     )
121     plt.title(title)
122     plt.xlabel("Component 1")
123     plt.ylabel("Component 2")
124     plt.colorbar(label="Class (0=Benign, 1=Malignant)")
125     plt.show()
126
127 # Plot PCA, t-SNE, and UMAP embeddings
128 plot_2d(X_pca, "PCA of Tumor Attributes")
129 plot_2d(X_tsne, "t-SNE of Tumor Attributes")
130 plot_2d(X_umap, "UMAP of Tumor Attributes")
131
```

Output

```
In [34]: %unfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/PCA_t-SNE_UMAP.py' --wdir
Explained variance ratio by PCA: [0.5116172 0.11405044]
C:\Users\polycarp.mugizi\AppData\Roaming\Python\Python313\site-packages\umap\umap.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
warn(
```

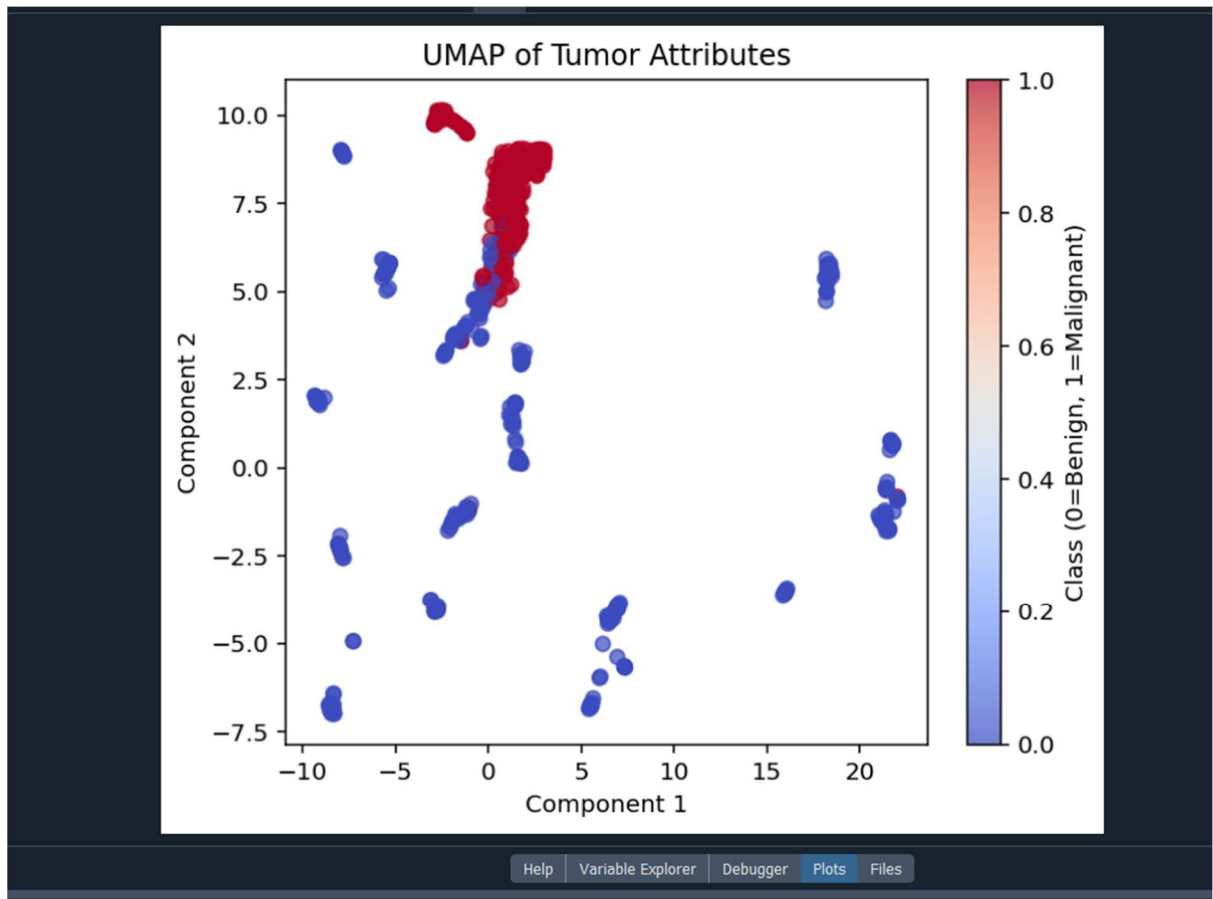




Help Variable Explorer Debugger **Plots** Files

history.py

```
unfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/scatterplot  
unfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/handle_mis
```

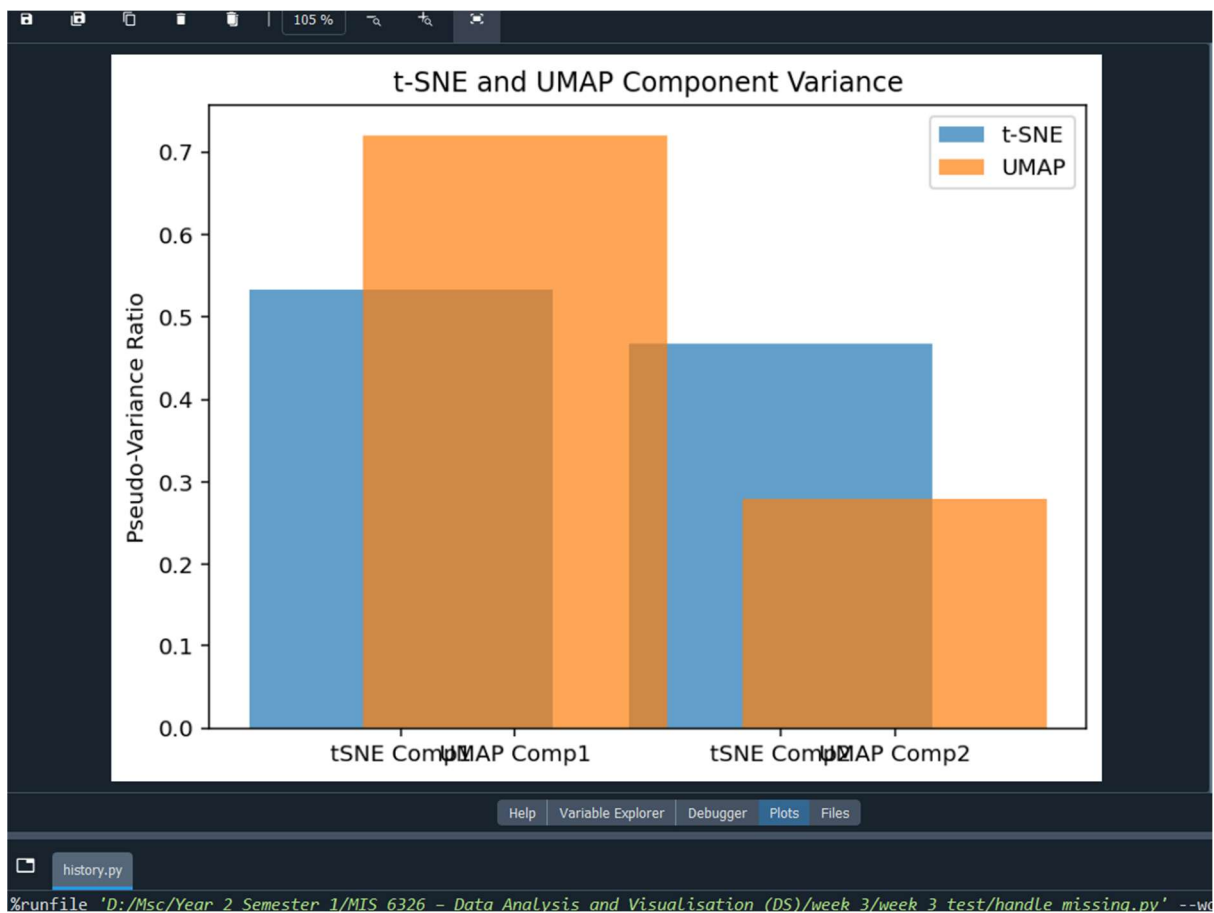
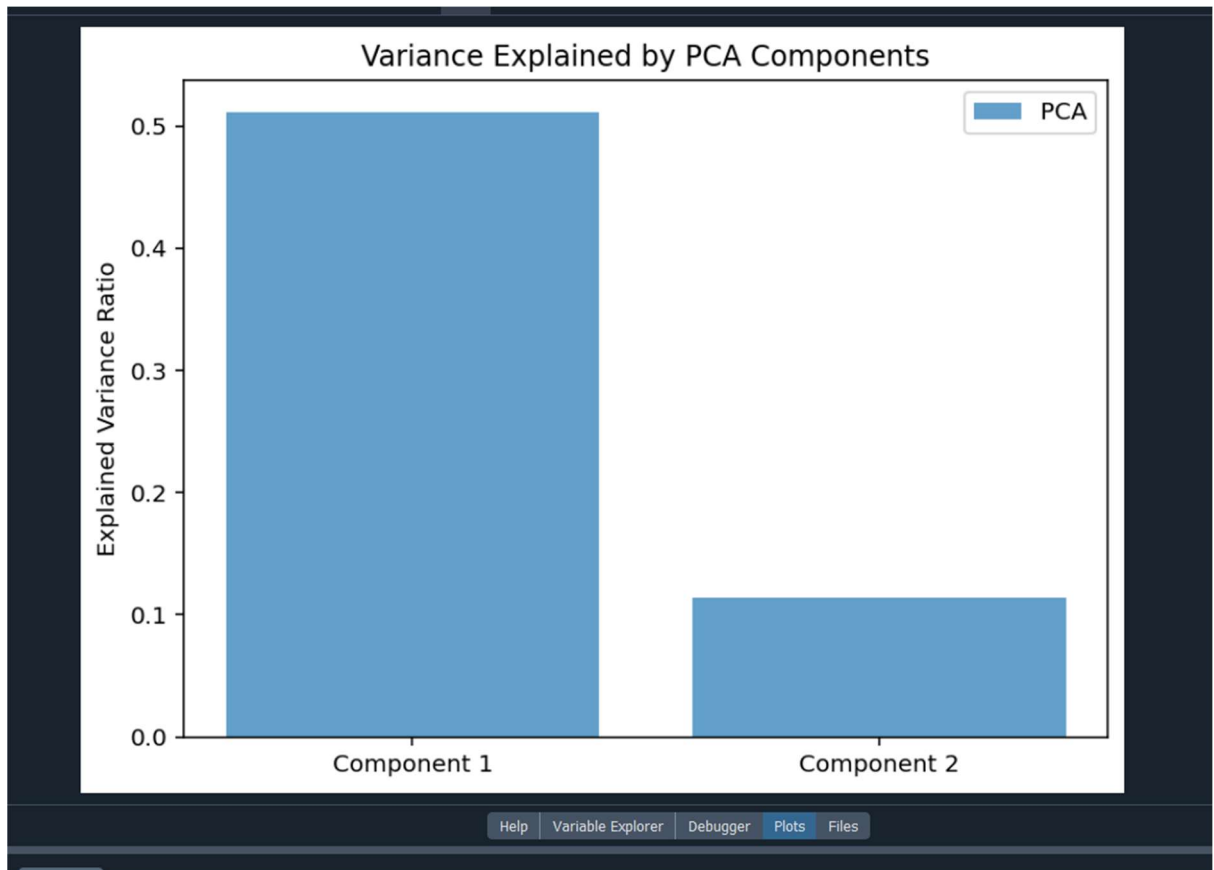


3. Second EDA (Post-Processing)

- Plot variance explained by the reduction methods (PCA, t-SNE, UMAP).

Code file: variance.py

```
In [15]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/variance.py' --wdir
PCA explained variance ratio: [0.5116172 0.11405044]
t-SNE pseudo-variance ratio: [0.5330572 0.4669428]
C:\Users\polycarp.mugizi\AppData\Roaming\Python\Python313\site-packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism.
  warn(
UMAP pseudo-variance ratio: [0.7206726 0.27932736]
```



- Feature importance visualization from Decision Tree.

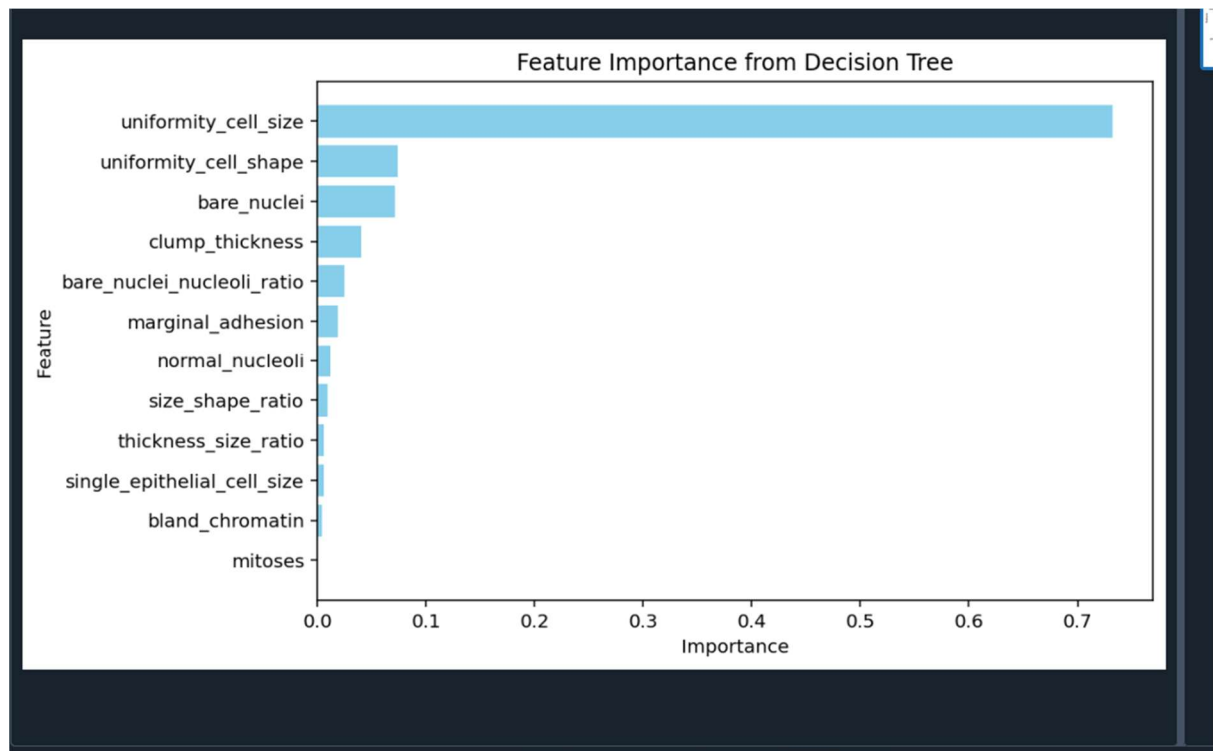
Code file: feature.py

```

feature.py* x
67 # 5. Prepare feature matrix X and target vector y
68 # -----
69 # X = all features except 'id' and 'class'
70 X = df.drop(columns=["id", "class"])
71
72 # y = target variable (0 = Benign, 1 = Malignant)
73 y = df["class"]
74
75 # -----
76 # 6. Normalize features (optional)
77 # -----
78 # Decision Trees do not require normalization, but we scale to keep consistent feature ranges
79 scaler = StandardScaler()
80 X_scaled = scaler.fit_transform(X)
81
82 # -----
83 # 7. Train Decision Tree Classifier
84 # -----
85 # Create a Decision Tree model
86 tree = DecisionTreeClassifier(random_state=42)
87
88 # Fit the model to the scaled feature matrix and target
89 tree.fit(X_scaled, y)
90
91 # -----
92 # 8. Extract feature importance
93 # -----
94 # Decision Tree assigns an importance score to each feature
95 # Higher score → feature contributes more to classification
96 importances = tree.feature_importances_
97
98 # Create a DataFrame to make plotting easier
99 feature_importance_df = pd.DataFrame({
100     "feature": X.columns,      # feature names
101     "importance": importances  # importance scores
102 }).sort_values(by="importance", ascending=False) # sort descending
103
104 # -----
105 # Plot feature importance
106 # -----
107 plt.figure(figsize=(8,5))
108
109 # Horizontal bar chart for readability
110 plt.barh(feature_importance_df["feature"], feature_importance_df["importance"], color='skyblue')
111
112 # Invert y-axis to have highest importance on top
113 plt.gca().invert_yaxis()
114
115 # Labels and title
116 plt.xlabel("Importance")
117 plt.ylabel("Feature")
118 plt.title("Feature Importance from Decision Tree")
119
120 # Display the plot
121 plt.show()
122

```

Output



4. ML Modeling: Train Naïve Bayes, k-NN.
Code file:

```

84 # 6. Split dataset into training and testing sets
85 # -----
86 # 80% training, 20% testing, stratify=y ensures class balance is preserved
87 # random_state=42 ensures reproducibility
88 X_train, X_test, y_train, y_test = train_test_split(
89     X, y, test_size=0.2, random_state=42, stratify=y
90 )
91
92 # -----
93 # 7. Normalize features
94 # -----
95 # Important for k-NN because it is distance-based
96 # Naïve Bayes can also benefit from normalized features
97 scaler = StandardScaler()
98
99 # Fit scaler on training data and transform
100 X_train_scaled = scaler.fit_transform(X_train)
101
102 # Transform test data using the same scaler
103 X_test_scaled = scaler.transform(X_test)
104
105 # -----
106 # 8a. Train Gaussian Naïve Bayes
107 # -----
108 # GaussianNB assumes features are normally distributed
109 nb_model = GaussianNB()
110
111 # Fit the Naïve Bayes model to the training data
112 nb_model.fit(X_train_scaled, y_train)
113
114 # Predict on test data
115 y_pred_nb = nb_model.predict(X_test_scaled)
116
117 # -----
118 # k-NN classifies based on the majority class of the k nearest neighbors
119 knn_model = KNeighborsClassifier(n_neighbors=5) # k=5
120
121 # Fit k-NN model to training data
122 knn_model.fit(X_train_scaled, y_train)
123
124 # Predict on test data
125 y_pred_knn = knn_model.predict(X_test_scaled)
126
127 # -----
128 # 9. Evaluate models
129 # -----
130
131 def evaluate_model(y_true, y_pred, model_name):
132     """
133     Prints evaluation metrics for a model
134     y_true: true labels
135     y_pred: predicted labels
136     model_name: string name of the model
137     """
138     print(f"=== {model_name} ===")
139
140     # Accuracy score
141     print("Accuracy:", accuracy_score(y_true, y_pred))
142
143     # Confusion matrix shows true positives, true negatives, false positives, false negatives
144     print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
145
146     # Classification report shows precision, recall, f1-score for each class
147     print("Classification Report:\n", classification_report(y_true, y_pred))
148     print("\n")
149
150 # Evaluate Gaussian Naïve Bayes
151 evaluate_model(y_test, y_pred_nb, "Gaussian Naïve Bayes")
152
153 # Evaluate k-Nearest Neighbors
154 evaluate_model(y_test, y_pred_knn, "k-Nearest Neighbors (k=5)")
155
156

```

Output

```
Console 3/A x
In [2]: %runfile 'D:/Msc/Year 2 Semester 1/MIS 6326 - Data Analysis and Visualisation (DS)/week 3/week_3_test/ml_modeling.py' --wdir
=== Gaussian Naïve Bayes ===
Accuracy: 0.9635036496350365
Confusion Matrix:
[[85  4]
 [ 1 47]]
Classification Report:
      precision    recall  f1-score   support

     0       0.99     0.96     0.97         89
     1       0.92     0.98     0.95         48

 accuracy          0.96         137
 macro avg       0.95     0.97     0.96         137
 weighted avg    0.96     0.96     0.96         137

=== k-Nearest Neighbors (k=5) ===
Accuracy: 0.9562043795620438
Confusion Matrix:
[[85  4]
 [ 2 46]]
Classification Report:
      precision    recall  f1-score   support

     0       0.98     0.96     0.97         89
     1       0.92     0.96     0.94         48

 accuracy          0.96         137
 macro avg       0.95     0.96     0.95         137
 weighted avg    0.96     0.96     0.96         137

In [3]:
```

5. Evaluation: Accuracy, Sensitivity/Specificity, ROC-AUC, Calibration curves for probabilistic models.

Code file: evaluation.py

```

117
118 # -----
119 # 9. Evaluation function
120 # -----
121 def evaluate_model(y_true, y_pred, y_prob, model_name):
122     """
123     Evaluate classifier performance with:
124     - Accuracy
125     - Sensitivity (Recall for positive class)
126     - Specificity (Recall for negative class)
127     - Confusion matrix
128     - Classification report
129     - ROC-AUC
130     - Calibration curve
131     """
132     print(f"=== {model_name} ===")
133
134     # Accuracy: proportion of correctly classified samples
135     acc = accuracy_score(y_true, y_pred)
136     print("Accuracy:", round(acc,4))
137
138     # Confusion matrix: TN, FP, FN, TP
139     cm = confusion_matrix(y_true, y_pred)
140     tn, fp, fn, tp = cm.ravel()
141     print("Confusion Matrix:\n", cm)
142
143     # Sensitivity = TP / (TP + FN)
144     sensitivity = tp / (tp + fn)
145
146     # Specificity = TN / (TN + FP)
147     specificity = tn / (tn + fp)
148
149     print("Sensitivity (Recall):", round(sensitivity,4))
150     print("Specificity:", round(specificity,4))
151
152     # Detailed classification report (precision, recall, f1-score)
153     print("Classification Report:\n", classification_report(y_true, y_pred))
154
155     # ROC curve
156     fpr, tpr, thresholds = roc_curve(y_true, y_prob)
157     roc_auc = auc(fpr, tpr)
158     print("ROC-AUC:", round(roc_auc,4))
159
160     # Plot ROC curve
161     plt.figure(figsize=(6,5))
162     plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {roc_auc:.3f})')
163     plt.plot([0,1], [0,1], color='red', linestyle='--') # random classifier
164     plt.xlabel("False Positive Rate")
165     plt.ylabel("True Positive Rate")
166     plt.title(f"ROC Curve - {model_name}")
167     plt.legend()
168     plt.show()
169
170     # Calibration curve
171     prob_true, prob_pred = calibration_curve(y_true, y_prob, n_bins=10)
172     plt.figure(figsize=(6,5))
173     plt.plot(prob_pred, prob_true, marker='o', label=f'{model_name}')
174     plt.plot([0,1], [0,1], linestyle='--', color='gray') # perfect calibration
175     plt.xlabel("Mean Predicted Probability")
176     plt.ylabel("Fraction of Positives")
177     plt.title(f"Calibration Curve - {model_name}")
178     plt.legend()
179     plt.show()
180
181 # -----
182 # 10. Evaluate models
183 # -----
184 evaluate_model(y_test, y_pred_nb, y_prob_nb, "Gaussian Naïve Bayes")
185 evaluate_model(y_test, y_pred_knn, y_prob_knn, "k-Nearest Neighbors (k=5)")
186

```

Output

```

=== k-Nearest Neighbors (k=5) ===
Accuracy: 0.9562
Confusion Matrix:
[[85  4]
 [ 2 46]]
Sensitivity (Recall): 0.9583
Specificity: 0.9551
Classification Report:

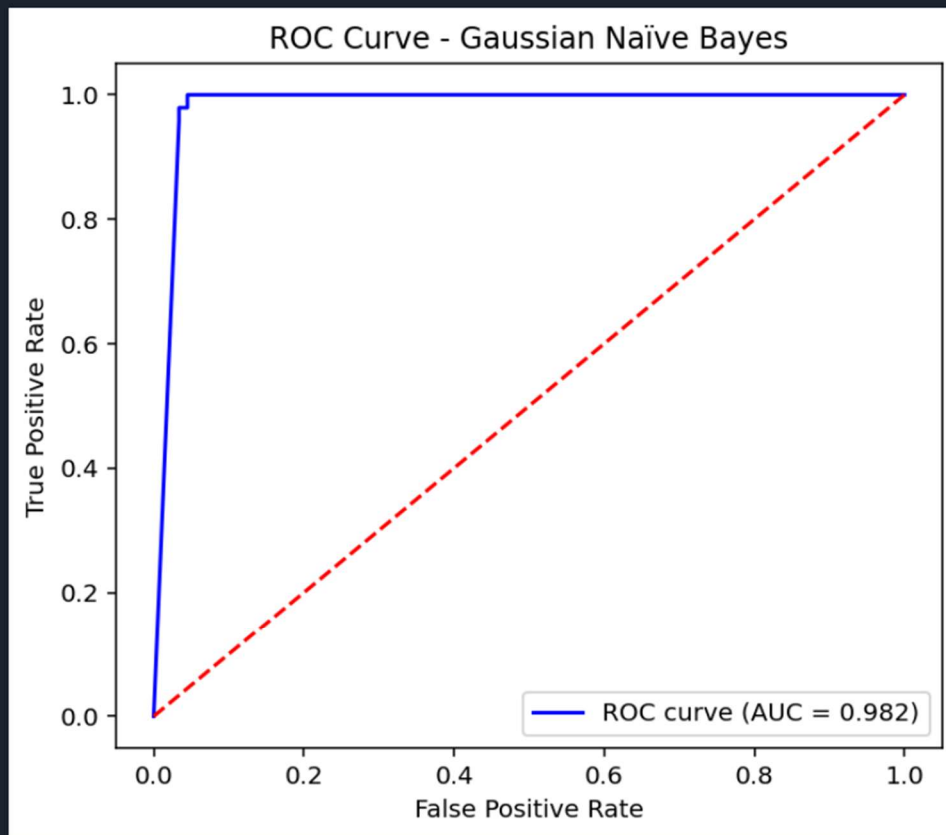
```

	precision	recall	f1-score	support
0	0.98	0.96	0.97	89
1	0.92	0.96	0.94	48
accuracy			0.96	137
macro avg	0.95	0.96	0.95	137
weighted avg	0.96	0.96	0.96	137

```

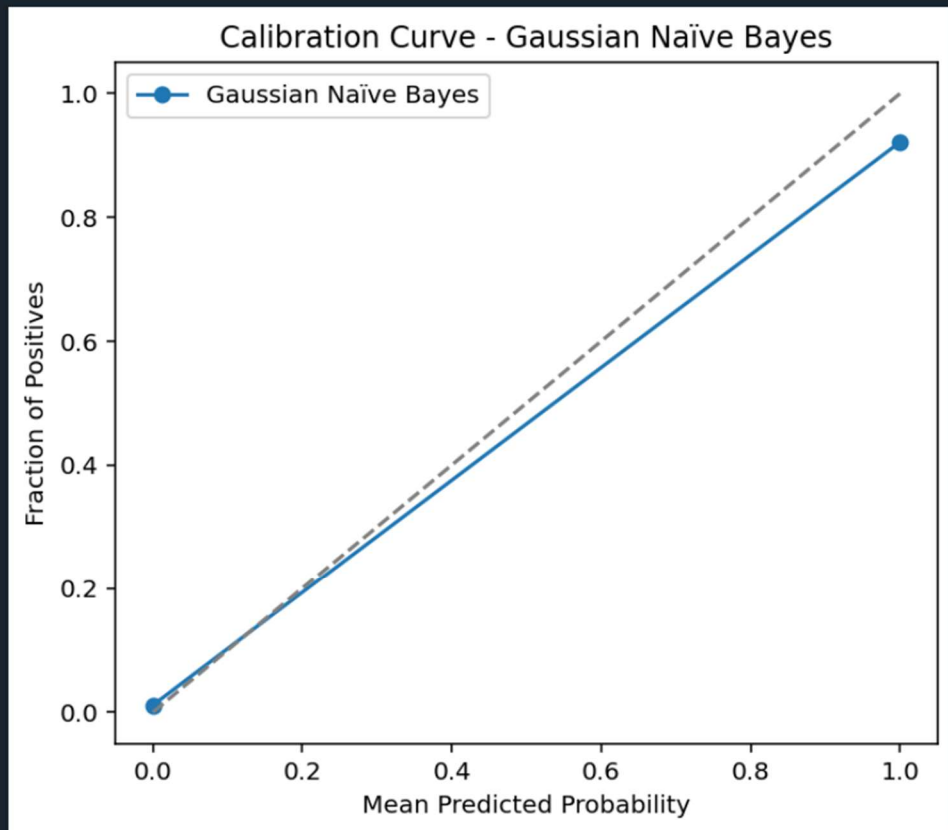
ROC-AUC: 0.9842

```

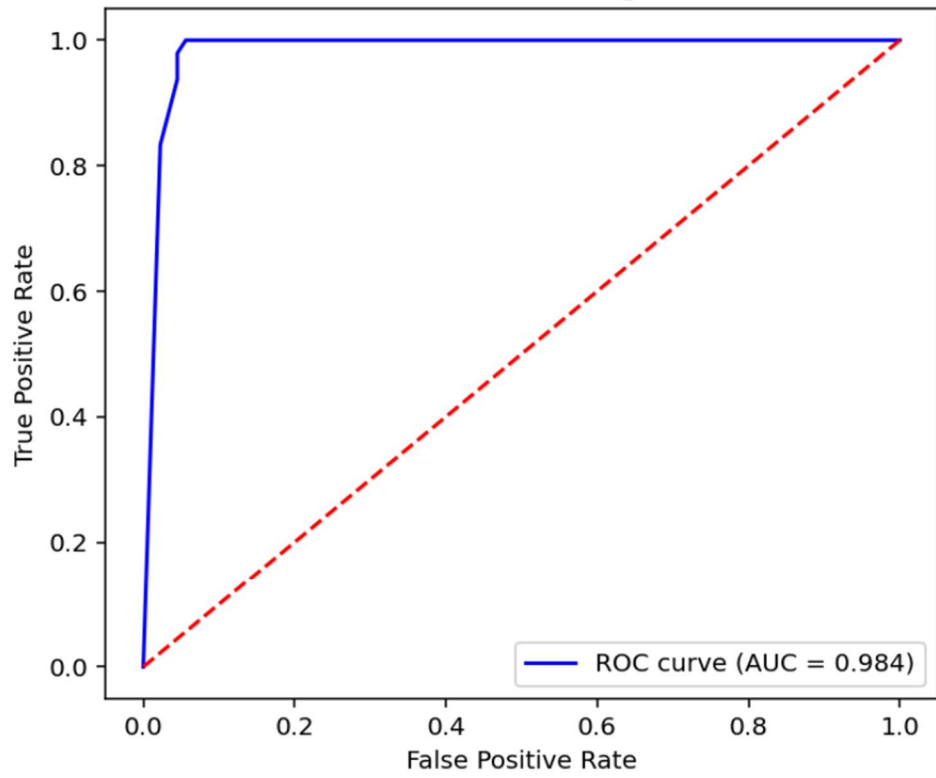


Help Variable Explorer Debugger Plots Files

Console 5/A x



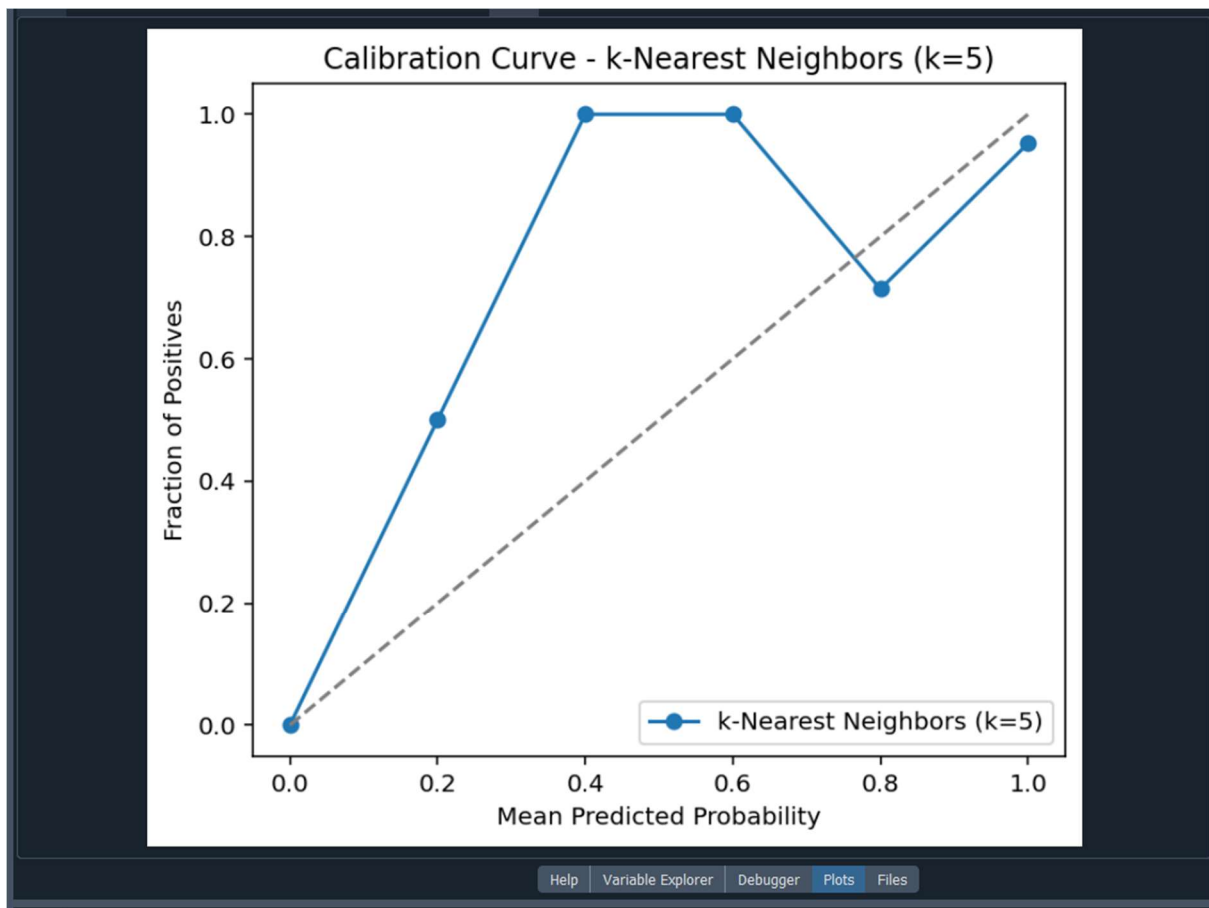
ROC Curve - k-Nearest Neighbors (k=5)



Help Variable Explorer Debugger Plots Files



Console 5/A ×



6. Interpretation

1. Features that best separate benign and malignant tumors

From feature importance (Decision Tree, RFE) and correlation with the target, the following features tend to be most predictive.

Feature	Reason it's important
Uniformity of cell size	Malignant tumors often have highly irregular cell sizes.
Uniformity of cell shape	Shape irregularity increases with malignancy.
Clump thickness	Malignant tumors tend to form denser clumps.
Bare nuclei	More prominent in malignant tumors.
Bland chromatin & normal nucleoli	Indicate cell division abnormalities.
Derived ratios (e.g., size_shape_ratio)	Capture combined abnormalities not evident from a single attribute.

- Also, Cell size, shape, and clump thickness are among the strongest single features.

- Boxplots of these features show that benign tumors are clustered at lower values and malignant tumors at higher values, with minimal overlap.
- Scatter plots of the top 3 features show malignant cases forming distinct clusters away from benign cases.

7. Implications of false negatives.

A false negative can occur if a malignant tumor is predicted as benign. Implications may include.

- i. Delayed treatment. The patient may not receive timely surgery, chemotherapy, or radiotherapy.
- ii. Disease progression. Malignant cells can metastasize, reducing survival probability.
- iii. Patient anxiety & trust. Misdiagnosis can undermine trust in the healthcare systems and personnel.